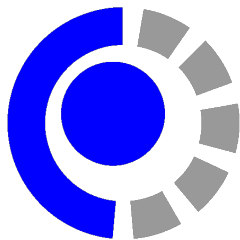


Tópicos Formales en Criptografía

Trabajo Práctico Obligatorio N°3

Adriano Mauricio Lusso FAI-2908
adriano.lusso@est.fi.uncoma.edu.ar

Primer cuatrimestre 2025



Facultad de Informática
UNIVERSIDAD NACIONAL DEL COMAHUE



1. Ejercicio 1

Se define un esquema de encriptación (Gen, Enc, Dec) basado en operaciones de desplazamiento modular y permutación de posiciones. A continuación se describen formalmente cada una de las tres funciones que componen el esquema.

Gen(n) Dado un parámetro $n \in \mathbb{N}$, que representa la longitud del mensaje a encriptar, el algoritmo Gen genera una clave $k = (s, \pi)$, donde $s \in \mathbb{Z}_{27}$ es un desplazamiento modular elegido uniformemente al azar, y π es una permutación aleatoria de los n caracteres del mensaje a encriptar. La clave se utiliza tanto para cifrar como para descifrar el mensaje.

Enc(k, m) Sea $m = (m_1, \dots, m_n) \in \mathbb{Z}_{27}^n$ un mensaje, representado como una secuencia de letras codificadas numéricamente, por ejemplo, A = 0, B = 1, ..., Z = 26. Para cifrar el mensaje bajo la clave $k = (s, \pi)$, se procede de la siguiente manera:

1. Se realiza un desplazamiento modular a cada símbolo del mensaje: $m'_i = (m_i + s) \bmod 27$.
2. Se aplica la permutación π sobre la secuencia desplazada m' .

El mensaje cifrado resultante es $c = (c_1, \dots, c_n) \in \mathbb{Z}_{27}^n$.

Dec(k, c) Dado un mensaje cifrado $c = (c_1, \dots, c_n)$ y la clave $k = (s, \pi)$, se procede a descifrar de la siguiente manera:

1. Se aplica la permutación inversa π^{-1} para recuperar el mensaje únicamente desplazado m' .
2. Se invierte el desplazamiento modular tal que $m_i = (m'_i - s) \bmod 27$.

El resultado es el mensaje original $m = (m_1, \dots, m_n)$.

Correctitud. Probamos que el esquema es correcto, es decir, que $\text{Dec}(k, \text{Enc}(k, m)) = m$ para toda clave k generada por Gen y todo mensaje $m \in \mathbb{Z}_{27}^n$.

Sea $k = (s, \pi)$ y $m = (m_1, \dots, m_n)$. Al cifrar, obtenemos $m'_i = (m_i + s) \bmod 27$ y luego c_i al aplicar la permutación. Las operaciones de desplazamiento modular y permutación son reversibles a partir de la clave k . Por lo tanto, al descifrar, se recupera m'_i aplicando la permutación inversa π^{-1} , y luego el desplazamiento inverso $m_i = (m'_i - s) \bmod 27$. Por lo tanto, el mensaje original se recupera correctamente.

2. Ejercicio 2

En el caso 2, en el que la clave k esté compuesta exclusivamente por unos, el cifrado se reduce a la negación bit a bit del mensaje:

$$\text{Enc}_k(m) = k \oplus m = 1^\ell \oplus m = \neg m,$$

y el descifrado se realiza aplicando nuevamente la misma operación:

$$\text{Dec}_k(c) = c \oplus k = \neg m \oplus 1^\ell = m.$$

Bajo estas condiciones, la propiedad de correctitud se mantiene. Además, dado que la clave se genera uniformemente al azar en el espacio $\{0,1\}^\ell$, la probabilidad de que $k = 1^\ell$ es $2^{-\ell}$, la cual es pequeña para valores grandes de ℓ . Debido a esto, con un espacio de claves lo suficientemente grande y la clave seleccionada uniformemente al azar, un atacante que desconozca la clave no puede saber ni explotar el hecho de que la clave sea exactamente 1^ℓ , y por lo tanto esta situación no compromete la seguridad del esquema. La vulnerabilidad existiría únicamente si la clave fuera fija o predecible.

En el caso 4, la clave es el complemento bit a bit del mensaje, es decir, $k = \neg m$. En este caso, el texto cifrado resulta ser $c = k \oplus m = \neg m \oplus m = 1^\ell$. Esto causa un gran problema. En primer lugar, todos los mensajes cifrados resultan en el mismo valor, lo que revela inmediatamente al atacante que se está utilizando la clave $k = \neg m$. Además, la clave ya no es independiente del mensaje, lo que viola la definición de un esquema de cifrado seguro. A pesar de esto, la correctitud se conserva, dado que $c = 1^\ell$ y $k = \neg m$, al aplicar la operación de descifrado se obtiene $m = c \oplus k = 1^\ell \oplus \neg m = m$.

3. Ejercicio 3

(a) Diferencias entre MAC y firma digital. Tanto MAC como las firmas digitales son mecanismos criptográficos utilizados para garantizar la integridad y autenticidad de los mensajes transmitidos. Sin embargo, difieren en su funcionamiento y en los escenarios en los que se aplican.

Un esquema MAC se basa en una clave secreta compartida entre el emisor y el receptor. Para enviar un mensaje, el emisor calcula un valor de autenticación, denominado MAC, aplicando un algoritmo criptográfico específico que combina la clave secreta y el contenido del mensaje. Este valor se envía junto con el mensaje original. Al recibirlo, el receptor, que también posee la misma clave secreta, puede calcular el MAC sobre el mensaje recibido y compararlo con el MAC recibido para verificar que el mensaje no ha sido alterado y que proviene de alguien que conoce la clave secreta. Debido a que ambas partes comparten la misma clave, el esquema MAC no permite a un tercero externo verificar quién generó el mensaje ni evitar que alguna de las partes niegue haberlo enviado. Este mecanismo es adecuado en entornos donde existe confianza mutua o canales seguros para la distribución y almacenamiento de claves secretas compartidas.

Por otro lado, una firma digital utiliza criptografía asimétrica, que involucra un par de claves. Una de las claves es privada y sólo la conoce el firmante, mientras que la otra es una clave pública que puede ser conocida por cualquier persona. El emisor genera la firma digital aplicando un algoritmo de firma sobre el mensaje usando su clave privada. Esta firma se adjunta al mensaje y puede ser verificada por cualquier tercero mediante la clave pública. La verificación confirma tanto la integridad del mensaje como la autenticidad del firmante, ya que sólo quien posee la clave privada pudo haber generado esa firma válida. Además, las firmas digitales permiten que el firmante no puede negar posteriormente haber firmado el mensaje, lo que es fundamental para aplicaciones legales. Este esquema es útil en entornos abiertos donde los participantes no necesariamente confían mutuamente, pero requieren garantías públicas sobre la autoría y autenticidad.

(b) Protocolos aplicables para firmas digitales. No todos los protocolos criptográficos sirven para implementar firmas digitales. Para que un protocolo sea adecuado, debe cumplir con ciertos requisitos:

- Uso de criptografía asimétrica.

- Seguridad frente a falsificación.
- Posibilidad de verificación pública de la firma sin comprometer la clave privada del firmante.

Por ejemplo, algoritmos como el RSA están diseñados específicamente para firmar digitalmente y permiten construir esquemas seguros y eficientes de firma. Sin embargo, otros protocolos simétricos como HMAC no pueden utilizarse para firmar en contextos donde se requiere verificación pública, ya que dependen de una clave secreta compartida.

(c) No repudio. El no repudio es una propiedad que asegura que un emisor no puede negar haber enviado un mensaje. Esta propiedad es esencial en entornos donde la responsabilidad legal de la transmisión de información es importante.

Las firmas digitales sí cumplen con el no repudio, ya que están asociadas de forma única al firmante mediante su clave privada, y cualquier persona puede verificar la autenticidad de la firma usando la clave pública correspondiente. Dado que sólo el firmante posee su clave privada, no puede negar válidamente haber generado la firma.

En cambio, los esquemas MAC no cumplen con esta propiedad. Al utilizar una clave secreta compartida entre el emisor y el receptor, cualquier parte que posea la clave podría haber generado el MAC. Por lo tanto, ante la presencia de un mensaje autenticado con MAC, no es posible probar estrictamente cuál de las dos partes lo generó, lo que impide garantizar el no repudio.

4. Ejercicio 4

El diseño del generador de tags presenta una gran vulnerabilidad, ya que el tag depende exclusivamente de la cantidad de unos en el mensaje, y no del mensaje completo. Por lo tanto, todos los mensajes que tengan la misma cantidad de unos producirán un tag con la misma longitud, y la operación XOR con la clave extendida será idéntica para todos ellos.

Consideremos un adversario A que realiza un ataque de mensaje elegido. El adversario elige un mensaje m_1 con una cierta cantidad de unos u_1 y obtiene el tag $t_1 = MAC_k(m_1)$ del sistema. Luego, A genera un nuevo mensaje m_2 tal que $CantidadUnos(m_2) = u_1$. Debido a la definición del MAC, el tag esperado para m_2 será idéntico a t_1 , es decir,

$$t_2 = MAC_k(m_2) = u_1 \oplus \text{extender}(k, rbu_1) = t_1,$$

Por lo tanto, el adversario puede forjar un par mensaje-tag válido (m_2, t_1) sin conocer la clave k .

Como caso concreto, consideremos que el adversario elige un mensaje $m_1 = 11010101$, que tiene $u_1 = 5$ unos. Supongamos que la clave $k = 01$ (desconocida para A). Entonces, la representación binaria de u_1 es 101, que ocupa $rbu = 3$ bits. La clave extendida para esta longitud es

$$\text{extender}(01, 3) = 010.$$

El tag resultante es entonces

$$t_1 = 101 \oplus 010 = 111.$$

Luego, A construye un nuevo mensaje $m_2 = 11111000$, que también tiene 5 unos, es decir, $u_2 = u_1 = 5$. El adversario puede forjar el tag para m_2 simplemente reutilizando t_1 , pues

$$t_2 = MAC_k(m_2) = 101 \oplus 010 = 111 = t_1.$$

De esta forma, sin conocer la clave k , el adversario puede producir un par (m_2, t_1) válido, que será aceptado por el verificador.

Este hecho implica que el esquema no provee autenticidad ni integridad contra adversarios con acceso a oráculos de MAC, violando la seguridad básica que debe tener un MAC. La dependencia exclusiva en la cantidad de unos ignora el contenido completo del mensaje, facilitando que este tipo de ataques sean exitosos.

5. Ejercicio 5

(a) **Definiciones formales** Sea $h : \{0, 1\}^* \rightarrow \{0, 1\}^n$ una función hash que mapea cadenas de longitud arbitraria a cadenas de longitud fija n .

- **Función de compresión:** Es una función que toma una entrada de tamaño mayor y la reduce a una salida de tamaño fijo y menor, sin que exista una función inversa eficiente. Formalmente, una función $f : \{0, 1\}^m \rightarrow \{0, 1\}^n$ con $m > n$ que comprime entradas largas en salidas más cortas.
- **Hash resistente a colisiones:** h es resistente a colisiones si es computacionalmente ineficiente encontrar dos entradas distintas $x \neq x'$ tales que $h(x) = h(x')$.
- **Hash resistente a segunda preimagen:** Dado un mensaje x , es computacionalmente costoso encontrar otro mensaje $x' \neq x$ tal que $h(x) = h(x')$. En esta definición la dificultad radica en encontrar una segunda preimagen específica para un mensaje dado, mientras que en la anterior se debe buscar el par (x, x') completo.
- **Hash resistente a preimagen:** Dado un valor y en el rango de h , es difícil encontrar cualquier mensaje x tal que $h(x) = y$. Es decir, es difícil invertir la función hash.

(b) **Análisis de la función**

¿Es compresora? Una función es compresora si su dominio es más grande que su codominio. En este caso, si consideramos que $x \in \mathbb{Z}_n$, entonces $g : \mathbb{Z}_n \rightarrow \mathbb{Z}_n$, es decir, dominio y codominio tienen el mismo tamaño. Por lo tanto, esta función no es necesariamente compresora, ya que no reduce la longitud de su entrada.

¿Es resistente a colisiones? La función $g(x) = x^2 \bmod n$ no es resistente a colisiones en general. Encontrar dos valores distintos $x \neq x'$ tales que $x^2 \equiv x'^2 \bmod n$ equivale a encontrar dos números diferentes que al elevarse al cuadrado dan el mismo resultado módulo n . Sin embargo, si no se conocen los factores de n , entonces encontrar una colisión es tan difícil como factorizar n , lo cual se considera computacionalmente costoso para valores grandes. Por esta razón, la función puede considerarse débilmente resistente a colisiones: es vulnerable si la factorización es conocida, pero segura si no lo es.

¿Es resistente a segunda preimagen? Esta propiedad exige que, dado un valor x , sea difícil encontrar un $x' \neq x$ tal que $g(x') = g(x)$. Dado que $g(x) = x^2 \bmod n$, y que si se conoce la factorización de n es posible calcular múltiples raíces cuadradas módulo n , esta función no es resistente a segunda preimagen en ese caso. Si la factorización no se conoce, el problema es tan difícil como la factorización de n .

¿Es resistente a preimagen? La resistencia a preimagen implica que, dado un valor y , es computacionalmente difícil encontrar un x tal que $x^2 \equiv y \bmod n$. Este es el problema del residuo cuadrático, conocido por ser tan difícil como la factorización de n . Por lo tanto, si la factorización de n es desconocida, la función g sí es resistente a preimagen.

(c) Demostración formal Supongamos que existe una función hash h que es resistente a colisiones pero no resistente a segunda preimagen. Es decir, dado un mensaje x , existe un $x' \neq x$ tal que $h(x) = h(x')$, y x' puede encontrarse eficientemente.

Sin embargo, encontrar tal x' para un x fijo es precisamente encontrar una colisión (x, x') en h , pues $x \neq x'$, $h(x) = h(x')$ y x ya está dado por la propia formulación de la propiedad. Esto genera un absurdo entre el hecho de que h es resistente a colisiones, pero a su vez que se puede encontrar una colisión para h eficientemente.

Por lo tanto, si h es resistente a colisiones, no existe un método eficiente para encontrar un x' distinto a un x dado que colisione con x . Esto implica que h también es resistente a segunda preimagen a través de una demostración por absurdo.

(d) Función One-Way La función $g(x) = x^2 \bmod n$, donde $n = pq$ con p y q primos grandes, es considerada una función unidireccional bajo en contexto actual en el no hay demostración respecto a $\mathbf{P} = \mathbf{NP}$. Esto significa que se puede calcular fácilmente, pero es difícil invertir: dado un valor y , encontrar un x tal que $x^2 \equiv y \bmod n$ es difícil si no se conocen los factores de n . Calcular raíces cuadradas módulo un número compuesto está relacionado con el problema de la factorización, que se considera computacionalmente costoso. Por eso, se cree que esta función es segura bajo el supuesto de que factorizar n es difícil.

Si se llegara a demostrar que $\mathbf{P} = \mathbf{NP}$, entonces todos los problemas que están en NP, como la factorización entera, podrían resolverse en tiempo polinómico. En ese caso, se podrían conocer los factores p y q , y entonces sería fácil calcular raíces cuadradas módulo n . Por lo tanto, $g(x) = x^2 \bmod n$ ya no sería una función unidireccional.

6. Ejercicio 6

CORRECTITUD Queremos demostrar que al finalizar el protocolo, tanto Alice como Bob comparten la misma clave secreta. Para ello, se analiza paso a paso el protocolo propuesto, utilizando las propiedades de la operación XOR:

1. $x \oplus 0 = x$
2. $x \oplus x = 0$
3. $x \oplus y = y \oplus x$

Paso 1: Alice elige de forma uniforme dos valores $k, e \in \{0, 1\}^n$, y calcula $c = k \oplus e$. Luego, envía c a Bob.

Paso 2: Bob elige de forma uniforme dos valores $a, b \in \{0, 1\}^n$, y calcula $d = a \oplus c \oplus b = a \oplus (k \oplus e) \oplus b$. Luego, envía d a Alice.

Paso 3: Alice recibe d y calcula $f = d \oplus e$. Reemplazando d , tenemos:

$$f = (a \oplus k \oplus e \oplus b) \oplus e$$

Aplicando las propiedades 3, 2 y 1, se obtiene:

$$f = a \oplus k \oplus b$$

Alice envía f a Bob.

Paso 4: Alice guarda como clave el valor k . Bob, por su parte, calcula su clave como $f \oplus a \oplus b$. Reemplazando f , tenemos:

$$f \oplus a \oplus b = (a \oplus k \oplus b) \oplus a \oplus b$$

Aplicando las propiedades 3, 2 y 1:

$$(a \oplus a) \oplus (b \oplus b) \oplus k = 0 \oplus 0 \oplus k = k$$

De esta forma, como ambos participantes terminan con la misma clave secreta k , se concluye que el protocolo es correcto.

SEGURIDAD Suponemos que Eve tiene acceso a los tres mensajes enviados públicamente durante el protocolo:

$$c = k \oplus e, \quad d = a \oplus c \oplus b, \quad f = d \oplus e$$

Vamos a intentar expresar k únicamente en función de los valores públicos (a, c, d, f) , que están disponibles para Eve.

Reescribimos f en función de c :

$$f = d \oplus e = (a \oplus c \oplus b) \oplus e = a \oplus (k \oplus e) \oplus b \oplus e$$

$$f = a \oplus k \oplus b \oplus e \oplus e = a \oplus k \oplus b,$$

por lo tanto,

$$f = a \oplus k \oplus b.$$

Esto implica:

$$f \oplus a \oplus b = k.$$

Es decir, si Eve pudiera obtener b , podría recuperar la clave k .

Para esto, se puede calcular:

$$c \oplus d = (k \oplus e) \oplus (a \oplus c \oplus b) = (k \oplus e) \oplus (a \oplus k \oplus e \oplus b) = a \oplus b.$$

De esta forma, Eve puede obtener el valor $a \oplus b$. Al concatenarlo con f , obtiene efectivamente la clave k .

7. Ejercicio 7

(a) Protocolo de Diffie–Hellman y definición de seguridad El protocolo de Diffie–Hellman es un esquema criptográfico fundamental que permite a dos partes, comúnmente denominadas Alice y Bob, establecer una clave secreta compartida a través de un canal de comunicación potencialmente inseguro, es decir, donde un atacante podría estar escuchando o interceptando la información transmitida. La característica principal de este protocolo es que no requiere que las partes compartan previamente información secreta, lo que lo hace especialmente útil en escenarios donde la distribución segura de claves es difícil o inviable.

Este protocolo se fundamenta en conceptos matemáticos relacionados al problema del logaritmo discreto en grupos cíclicos finitos. El problema del logaritmo discreto consiste en, dado un número primo grande p , un generador g de un grupo multiplicativo módulo p , y un valor $h = g^x \bmod p$, determinar el exponente x . Este problema es considerado computacionalmente costoso para valores suficientemente grandes de p , lo que garantiza la seguridad del protocolo.

Inicialmente, Alice y Bob acuerdan públicamente dos parámetros, un número primo p suficientemente grande y un generador g del grupo multiplicativo $\mathbb{Z}_p^*$ (que equivale a todos los enteros positivos menores que p y que son coprimos de p). Ambos valores pueden ser conocidos por cualquier tercero sin comprometer la seguridad del protocolo. A continuación, Alice selecciona de manera aleatoria y secreta un número x como su clave privada y calcula su valor público como $A = g^x \bmod p$, que envía a Bob a través del canal público. Paralelamente, Bob realiza una operación análoga, en la que escoge un número secreto y , calcula $B = g^y \bmod p$ y lo envía a Alice.

Al recibir estos valores, Alice utiliza el valor público de Bob para calcular la clave secreta compartida como $k = B^x \bmod p$, mientras que Bob hace lo propio calculando $k = A^y \bmod p$. Gracias a las propiedades de la exponenciación modular, ambos cálculos producen el mismo resultado, ya que

$$k = (g^y)^x \bmod p = (g^x)^y \bmod p = g^{xy} \bmod p.$$

Este valor k es la clave secreta que Alice y Bob podrán utilizar para cifrar y descifrar mensajes posteriores.

La seguridad de este intercambio se basa en que, aunque un atacante pueda interceptar y conocer los valores públicos p , g , A y B , no puede deducir el valor secreto k sin resolver el problema del logaritmo discreto, es decir, sin conocer x o y . Este problema es computacionalmente difícil cuando p es lo suficientemente grande y se eligen correctamente los parámetros, haciendo inviable para un atacante calcular k en un tiempo razonable con los recursos computacionales actuales.

Por último, es importante destacar que el protocolo por sí solo no provee autenticación, lo que significa que no garantiza la identidad de las partes involucradas en el intercambio, dejando abierta la posibilidad de ataques (man-in-the-middle). Por ello, en aplicaciones prácticas, Diffie–Hellman suele combinarse con mecanismos de autenticación para proteger la integridad y autenticidad del intercambio.

(b) Cálculo de la clave secreta compartida Dado $p = 17$, $g = 2$, secreto de Alice $x = 5$, y secreto de Bob $y = 2$:

Calculamos los valores públicos:

$$A = g^x \bmod p = 2^5 \bmod 17 = 32 \bmod 17 = 15,$$

$$B = g^y \bmod p = 2^2 \bmod 17 = 4.$$

Luego, Alice y Bob calculan la clave compartida:

$$k = B^x \bmod p = 4^5 \bmod 17 = 4.$$

$$k = A^y \bmod p = 15^2 \bmod 17 = 4.$$

Por lo tanto, la clave secreta compartida es $k = 4$.

(c) Ataque man-in-the-middle En un ataque man-in-the-middle, un atacante Mallory intercepta la comunicación entre Alice y Bob, haciéndose pasar simultáneamente por cada uno ante el otro sin que ellos lo detecten.

El ataque procede así:

1. Alice envía a Bob su valor público $A = g^x \bmod p$, pero Mallory lo intercepta y no se lo reenvía a Bob.
2. Mallory genera su propio secreto m , calcula $M = g^m \bmod p$ y se lo envía a Bob como si fuera Alice.
3. Bob responde con $B = g^y \bmod p$, que Mallory intercepta.
4. Mallory envía M a Alice como si fuera Bob.
5. Ahora, Alice y Mallory calculan una clave compartida $k_{AM} = M^x \bmod p$, mientras que Mallory y Bob calculan $k_{MB} = B^m \bmod p$.

De esta forma, Mallory puede descifrar y modificar todos los mensajes entre Alice y Bob, ya que ambos creen estar comunicándose directamente y usan claves compartidas distintas, cada una con Mallory.

La razón por la que este ataque es posible es la falta de autenticación en el protocolo original Diffie–Hellman. El protocolo solo garantiza confidencialidad si no hay un atacante intermediario, pero no provee mecanismos para verificar la identidad de las partes, lo que permite a Mallory insertarse sin ser detectado.